

Lab3-demo

Queue ADT with Circular Array Implementation

Yuhan Wang

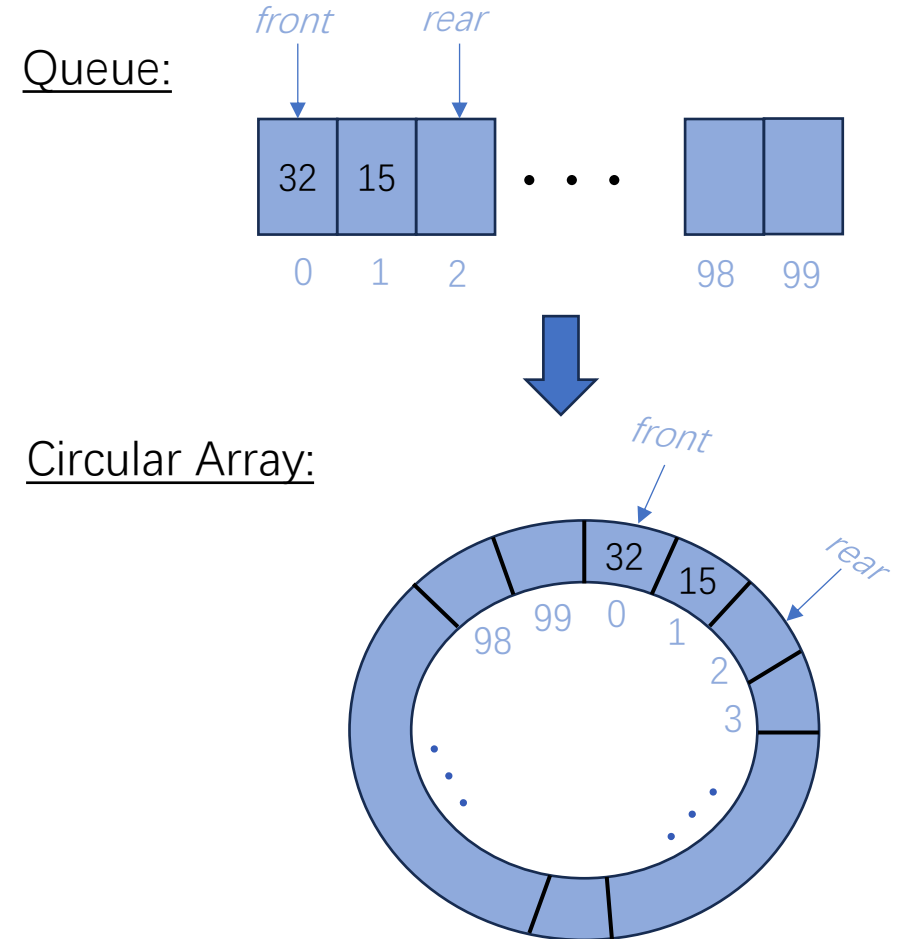
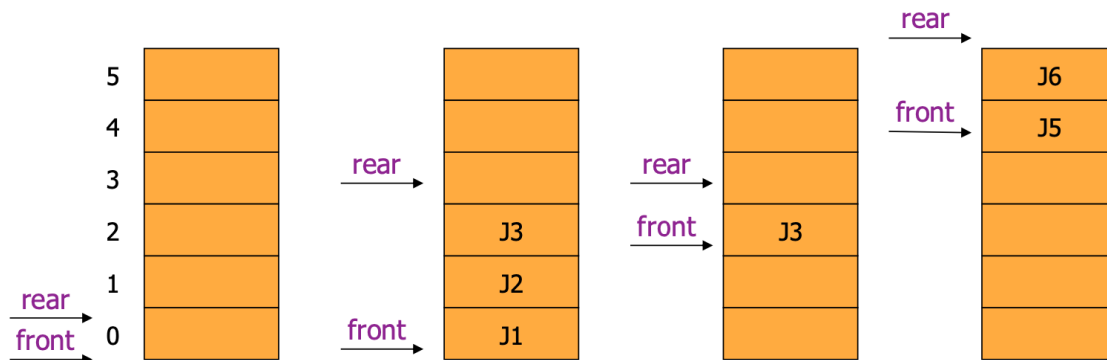
wy023@ie.cuhk.edu.hk

Oct. 9, 2024

Understand the Structure

- In Lab3-q1.h file:

```
#define MAX_SIZE 100
struct queue{
    int data[MAX_SIZE];
    int front; // The front of the queue
    int rear; // The rear of the queue,
              // the next position where a new item
              // can enqueue if the queue is not full.
    int size; // The size of the queue
};
typedef struct queue Queue;
```



MAX_SIZE = 100
front = 0
rear = 2
size = 2

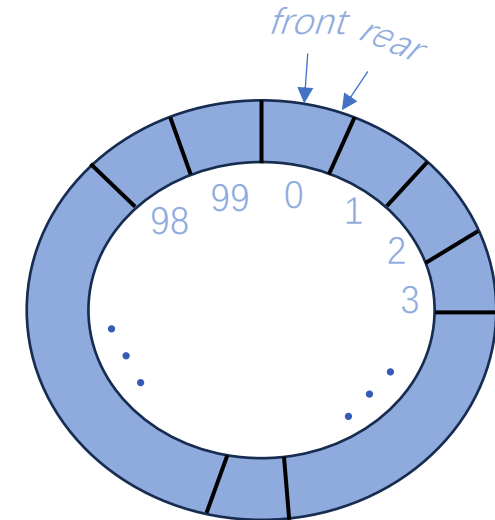
Initialize the Queue

- Initialize a new empty queue

```
/* Initialize a new empty queue #1 */  
// An empty queue has 0 values for  
front, rear and size  
void queue_init(Queue** q) {  
    Queue* tmpq = malloc(sizeof(Queue));  
    queue_make_empty(tmpq);  
    *q = tmpq;  
}
```

```
/* Make the queue empty #2 */  
/* An empty queue has 0 values for  
front, rear and size */  
void queue_make_empty(Queue* q){  
    q->front = 0;  
    q->rear = 0;  
    q->size = 0;  
}
```

malloc a block of memory for
the **Queue**



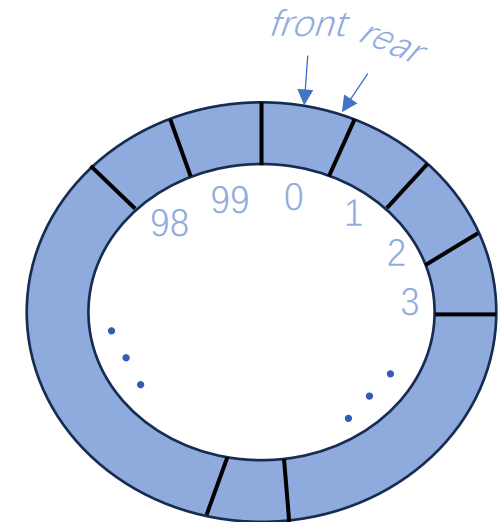
and deal with the member variables
properly:

front = 0
rear = 0
size = 0

Initialize the Queue

■ Size function

```
/* Return the size of the queue #3 */  
int queue_size(Queue* q) {  
    return q->size;  
}
```



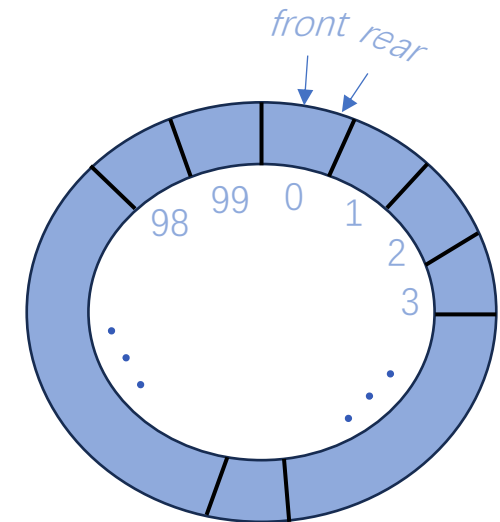
front = 0
rear = 0
size = 0

Operations on the Queue

■ Enqueue

```
/* If the queue is not full, enqueue
the value x to the rear of the queue #6
*/
void queue_enqueue(Queue* q, int x){
    if (queue_is_full(q)){
        return;
    }
    q->size++;
    q->data[q->rear] = x;
    q->rear = (q->rear + 1) % MAX_SIZE;
}
```

```
/ Return 1 if full, return 0 if not full
*/
int queue_is_full(Queue* q){
    if (q->size == MAX_SIZE) return 1;
    return 0;
}
```



front = 0
rear = 0
size = 0

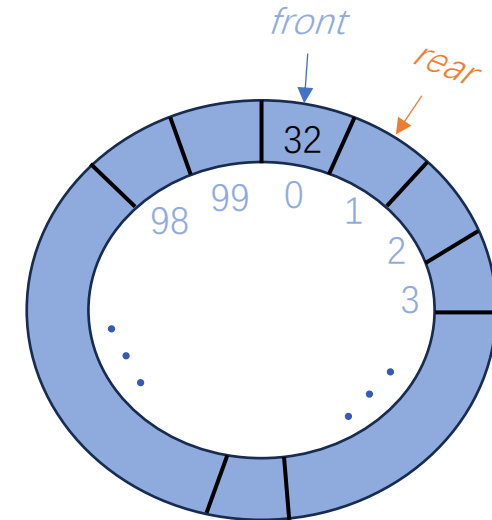
Operations on the Queue

- When we call the function

```
/* If the queue is not full, enqueue
the value x to the rear of the queue #6
*/
void queue_enqueue(Queue* q, int x){
    if (queue_is_full(q)){
        return;
    }
    q->size++;
    q->data[q->rear] = x;
    q->rear = (q->rear + 1) % MAX_SIZE;
}
```

```
/ Return 1 if full, return 0 if not full
*/
int queue_is_full(Queue* q){
    if (q->size == MAX_SIZE) return 1;
    return 0;
}
```

`queue_enqueue(q, 32)`



front = 0
rear = 1
size = 1

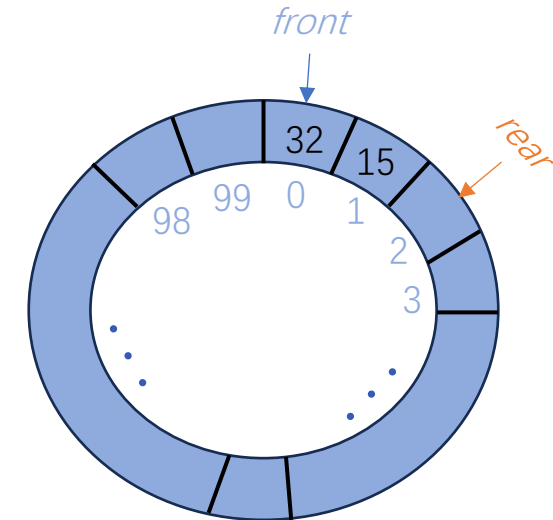
Operations on the Queue

- When we call the function

```
/* If the queue is not full, enqueue
the value x to the rear of the queue #6
*/
void queue_enqueue(Queue* q, int x){
    if (queue_is_full(q)){
        return;
    }
    q->size++;
    q->data[q->rear] = x;
    q->rear = (q->rear + 1) % MAX_SIZE;
}
```

```
/ Return 1 if full, return 0 if not full
*/
int queue_is_full(Queue* q){
    if (q->size == MAX_SIZE) return 1;
    return 0;
}
```

`queue_enqueue(q, 15)`



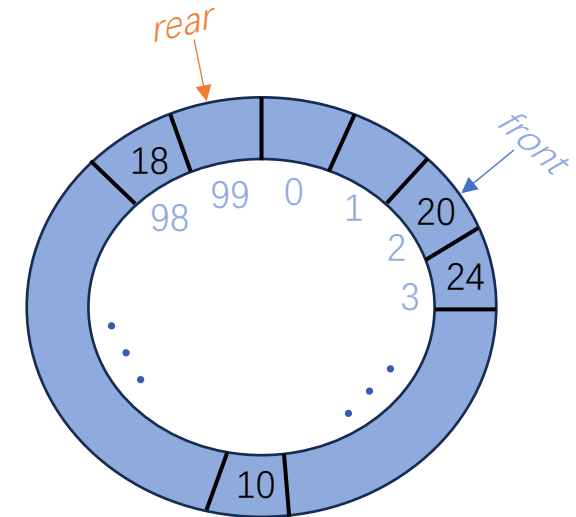
front = 0
rear = 2
size = 2

Operations on the Queue

- When it comes to the tail, remember this is a circular...

```
/* If the queue is not full, enqueue
the value x to the rear of the queue #6
*/
void queue_enqueue(Queue* q, int x){
    if (queue_is_full(q)){
        return;
    }
    q->size++;
    q->data[q->rear] = x;
    q->rear = (q->rear + 1) % MAX_SIZE;
}
```

```
/ Return 1 if full, return 0 if not full
*/
int queue_is_full(Queue* q){
    if (q->size == MAX_SIZE) return 1;
    return 0;
}
```



front = 2
rear = 99
size = 97

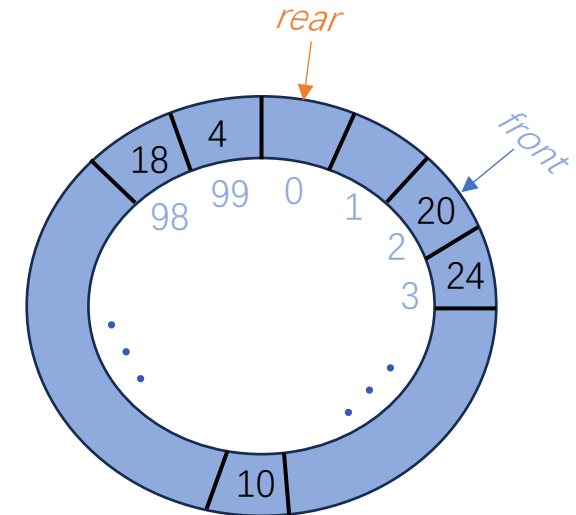
Operations on the Queue

- When it comes to the tail, remember this is circular...

```
/* If the queue is not full, enqueue
the value x to the rear of the queue #6
*/
void queue_enqueue(Queue* q, int x){
    if (queue_is_full(q)){
        return;
    }
    q->size++;
    q->data[q->rear] = x;
    q->rear = (q->rear + 1) % MAX_SIZE;
}
```

```
/ Return 1 if full, return 0 if not full
*/
int queue_is_full(Queue* q){
    if (q->size == MAX_SIZE) return 1;
    return 0;
}
```

`queue_enqueue(q, 4)`



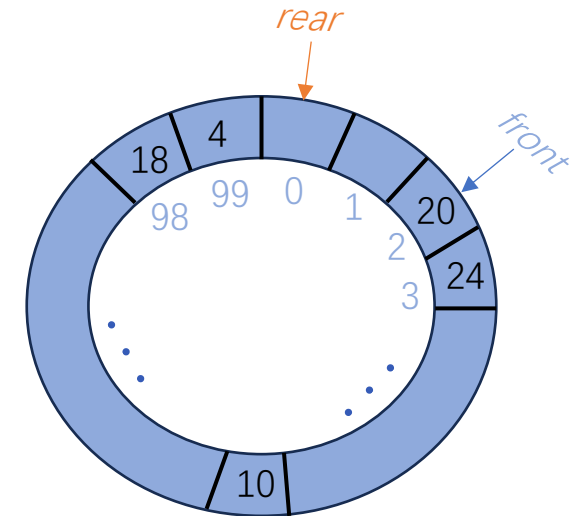
front = 2
rear = 0
size = 98

Operations on the Queue

- Same with dequeue

```
/* If q is not empty, dequeue and return the
front of the queue #7 */
/* If the queue is empty, return INT_MAX */
int queue_dequeue(Queue* q){
    int x;
    if (queue_is_empty(q)){
        return INT_MAX;
    }
    q->size--;
    x = q->data[q->front];
    q->front = (q->front + 1) % MAX_SIZE;
    return x;
}
```

```
int queue_is_empty(Queue* q){
    if (q->size == 0) return 1;
    return 0;
}
```



front = 2
rear = 0
size = 98

Operations on the Queue

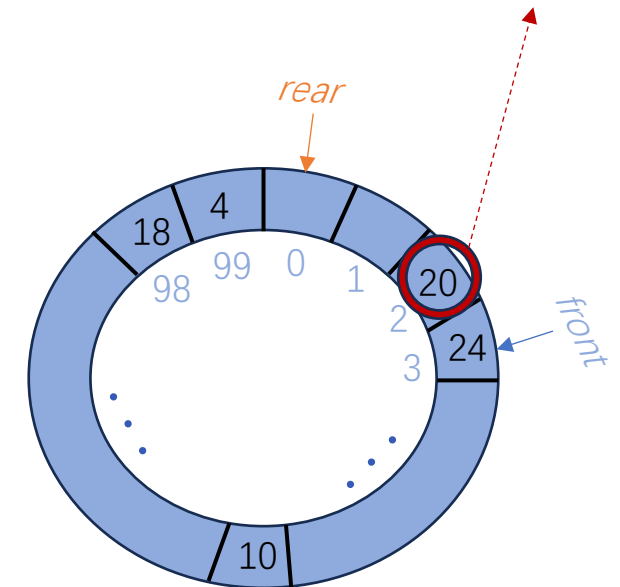
- Same with dequeue

```
/* If q is not empty, dequeue and return the front of the queue #7 */  
/* If the queue is empty, return INT_MAX */  
int queue_dequeue(Queue* q){  
    int x;  
    if (queue_is_empty(q)){  
        return INT_MAX;  
    }  
    q->size--;  
    x = q->data[q->front];  
    q->front = (q->front + 1) % MAX_SIZE;  
    return x;  
}
```

```
int queue_is_empty(Queue* q){  
    if (q->size == 0) return 1;  
    return 0;  
}
```

queue_dequeue(q)

Return 20



front = 3
rear = 0
size = 97

Operations on the Queue

■ Print function

```
/* Print the stack in the format #9 */
/* e.g. q: 0 1 2 3, where 0 is the front */
/* output: "(front) 0 1 2 3 (rear)" */
char* queue_print(Queue* q){
    int len, i, cnt;
    char *output;
    len = q->size*12+15;
    output = malloc(sizeof(char)*len);
    memset(output, 0, sizeof(char)*len);
    sprintf(output, "(front)");
    for (i = q->front, cnt = 0; cnt < q->size;
        i = (i+1)%MAX_SIZE, cnt++) {
        sprintf(output, "%s %d", output, q->data[i]);
    }
    sprintf(output, "%s (rear)", output);
    return output;
}
```

Operations on the Queue

■ Print function

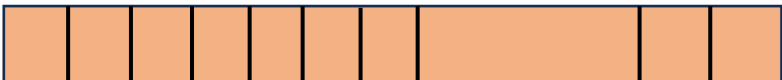
```
/* Print the stack in the format #9 */
/* e.g. q: 0 1 2 3, where 0 is the front */
/* output: "(front) 0 1 2 3 (rear)" */
char* queue_print(Queue* q){
    int len, i, cnt;
    char *output;
    len = q->size*12+15;
    output = malloc(sizeof(char)*len);
    memset(output, 0, sizeof(char)*len);
    sprintf(output, "(front)");
    for (i = q->front, cnt = 0; cnt < q->size;
        i = (i+1)
        >data[i])
        sprintf(output, "%d ", data[i]);
    return output;
}
```

Limits on Integer Constants

Constant	Meaning	Value
INT_MIN	Minimum value for a variable of type int .	-2147483647
INT_MAX	Maximum value for a variable of type int .	2147483647

malloc the memory for output string

output:



What is q->size*12+15?

"(front) 0 1 2 3 (rear)"

15: "(front) (rear)\0"

q->size: number of elements in queue

12: each element in queue is an **int**
+ space



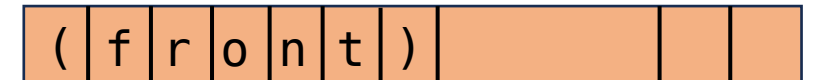
Operations on the Queue

■ Print function

```
/* Print the stack in the format #9 */
/* e.g. q: 0 1 2 3, where 0 is the front */
/* output: "(front) 0 1 2 3 (rear)" */
char* queue_print(Queue* q){
    int len, i, cnt;
    char *output;
    len = q->size*12+15;
    output = malloc(sizeof(char)*len);
    memset(output, 0, sizeof(char)*len);
    sprintf(output, "(front)");
    for (i = q->front, cnt = 0; cnt < q->size;
        i = (i+1)%MAX_SIZE, cnt++) {
        sprintf(output, "%s %d", output, q-
>data[i]);
    }
    sprintf(output, "%s (rear)", output);
    return output;
}
```

malloc the memory for output string

output:



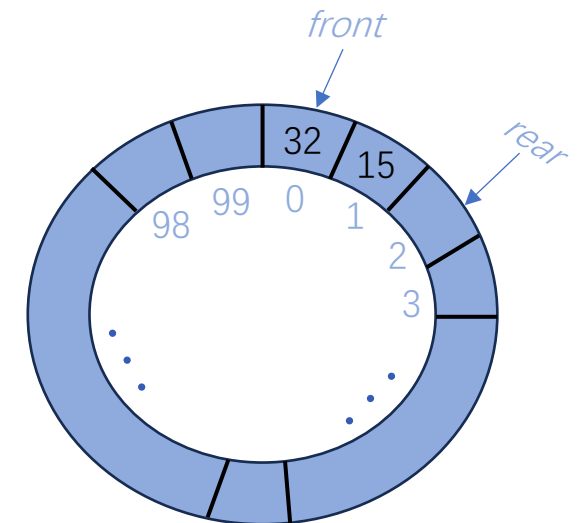
Operations on the Queue

■ Print function

```
/* Print the stack in the format #9 */
/* e.g. q: 0 1 2 3, where 0 is the front */
/* output: "(front) 0 1 2 3 (rear)" */
char* queue_print(Queue* q){
    int len, i, cnt;
    char *output;
    len = q->size*12+15;
    output = malloc(sizeof(char)*len);
    memset(output, 0, sizeof(char)*len);
    sprintf(output, "(front)");
    for (i = q->front, cnt = 0; cnt < q->size;
        i = (i+1)%MAX_SIZE, cnt++) {
        sprintf(output, "%s %d", output, q-
            >data[i]);
    }
    sprintf(output, "%s (rear)", output);
    return output;
}
```

malloc the memory for output string

output:



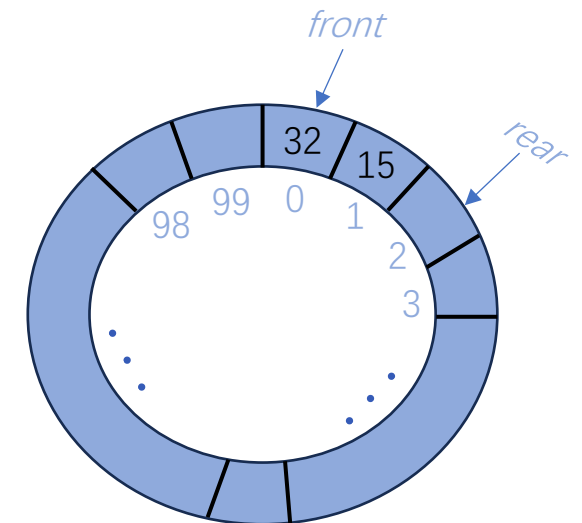
Operations on the Queue

■ Print function

```
/* Print the stack in the format #9 */
/* e.g. q: 0 1 2 3, where 0 is the front */
/* output: "(front) 0 1 2 3 (rear)" */
char* queue_print(Queue* q){
    int len, i, cnt;
    char *output;
    len = q->size*12+15;
    output = malloc(sizeof(char)*len);
    memset(output, 0, sizeof(char)*len);
    sprintf(output, "(front)");
    for (i = q->front, cnt = 0; cnt < q->size;
        i = (i+1)%MAX_SIZE, cnt++) {
        sprintf(output, "%s %d", output, q-
            >data[i]);
    }
    sprintf(output, "%s (rear)", output);
    return output;
}
```

malloc the memory for output string

output:



Free the Queue

■ Free function

```
/* Free the queue, if the queue is not NULL
#8 */
/* Assign NULL to q after free */
void queue_free(Queue** q) {
    if (*q != NULL) free(*q);
    *q = NULL;
}
```

```
/* Return a new empty queue #1 */
void queue_init(Queue** q) {
    Queue* tmpq = malloc(sizeof(Queue));
    queue_make_empty(tmpq);
    *q = tmpq;
}
```

Each time you **malloc** some memory,
there must be a **free** function in the future

Some hints on Q2

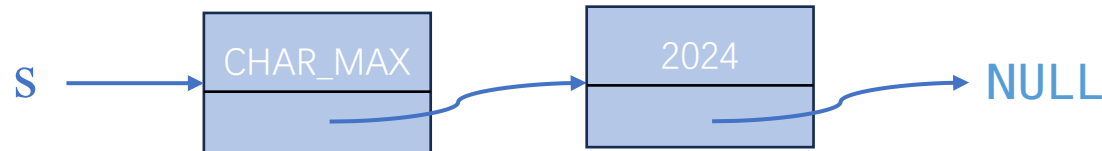
■ Q2: Stack ADT with Linked List Implementation

// An empty stack is data storing CHAR_MAX and next pointing to NULL



// Push the value x to the top of the stack

stack_push(s, 2024)



Lab3-q2.h

```
/* Cannot modify: struct  
node, Stack */  
struct node {  
    char data;  
    struct node *next;  
};  
typedef struct node Stack;
```

■ Check lecture slides for more details.